

Architecture Overview - E-commerce Cart System

Table of Contents

1. [System Context](#)
 2. [Architectural Views](#)
 3. [Technology Stack](#)
 4. [Design Decisions](#)
 5. [Quality Attributes](#)
 6. [Risk Assessment](#)
-

System Context

The E-commerce Cart System is a modern, scalable platform designed to handle online retail operations with a focus on user experience, performance, and maintainability. The system serves multiple user types including customers, employees, and suppliers within a unified architecture.

Core Business Capabilities

- **Product Management:** Catalog management, inventory control, and pricing
 - **User Management:** Customer registration, authentication, and profile management
 - **Order Processing:** Shopping cart, checkout, and order fulfillment
 - **Financial Operations:** Billing, invoicing, and payment processing
 - **Reporting & Analytics:** Business intelligence and operational insights
-

Architectural Views

1. Business View

The business view represents the organizational structure and business processes that the system supports.

Key Stakeholders

- **Customers:** End users who browse, purchase, and manage their orders
- **Employees:** Internal staff managing products, orders, and customer support
- **Suppliers:** External vendors providing products and inventory
- **Administrators:** System managers overseeing operations and configuration

Business Processes

1. **Customer Journey:** Registration → Browsing → Cart Management → Checkout → Order Tracking
2. **Inventory Management:** Supplier Integration → Stock Updates → Reorder Management
3. **Order Fulfillment:** Order Processing → Payment Verification → Shipping → Delivery Confirmation

2. Logical View

The logical view defines the system's components, their responsibilities, and interactions.

Core Components

Domain Layer

- **Product Domain:** Product catalog, categories, pricing, and inventory management
- **User Domain:** User profiles, authentication, authorization, and preferences
- **Order Domain:** Shopping cart, order processing, and fulfillment workflows
- **Financial Domain:** Billing, invoicing, and payment processing

Application Layer

- **Product Service:** Product catalog operations and inventory management
- **User Service:** User management, authentication, and profile operations
- **Order Service:** Order processing, cart management, and fulfillment
- **Payment Service:** Payment processing and financial operations
- **Notification Service:** Email, SMS, and in-app notifications

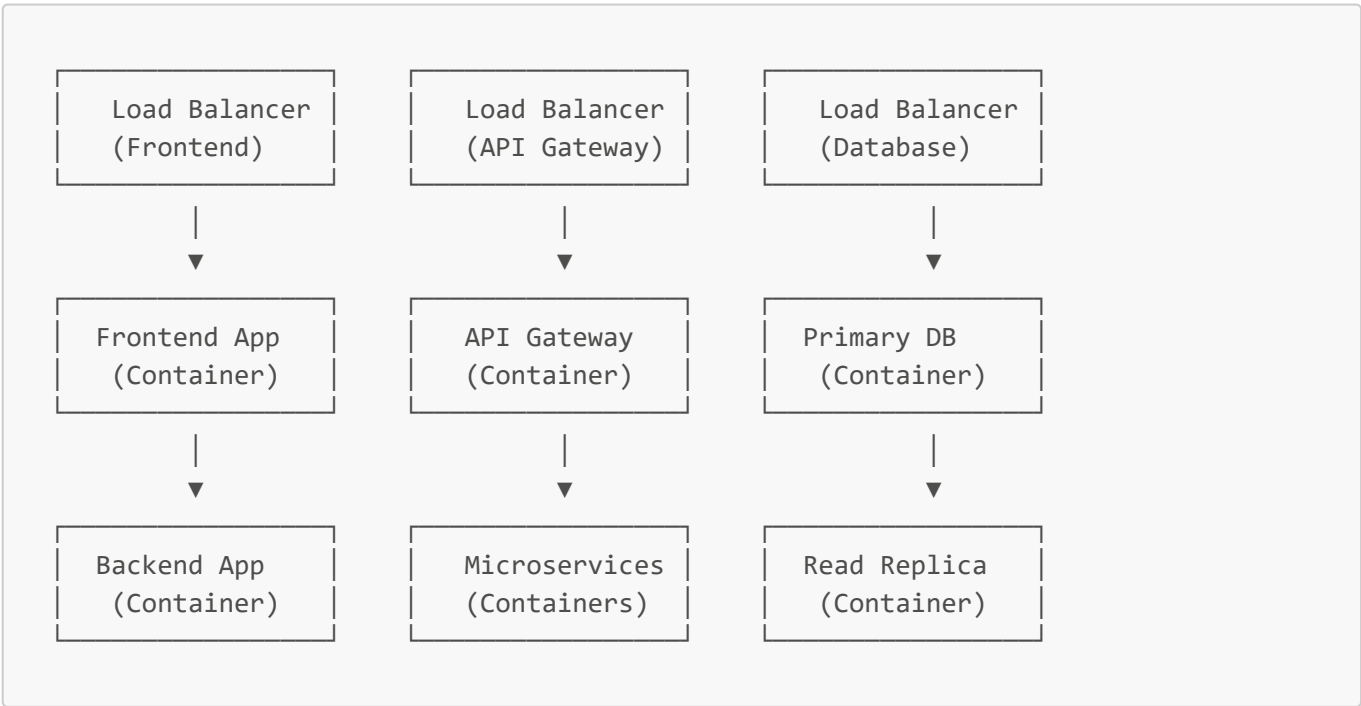
Infrastructure Layer

- **Data Access:** Repository implementations and data persistence
- **External Integrations:** Payment gateways, shipping providers, and third-party services
- **Security:** Authentication, authorization, and data protection
- **Monitoring:** Logging, metrics, and health checks

3. Physical View

The physical view describes the deployment architecture and infrastructure components.

Deployment Architecture



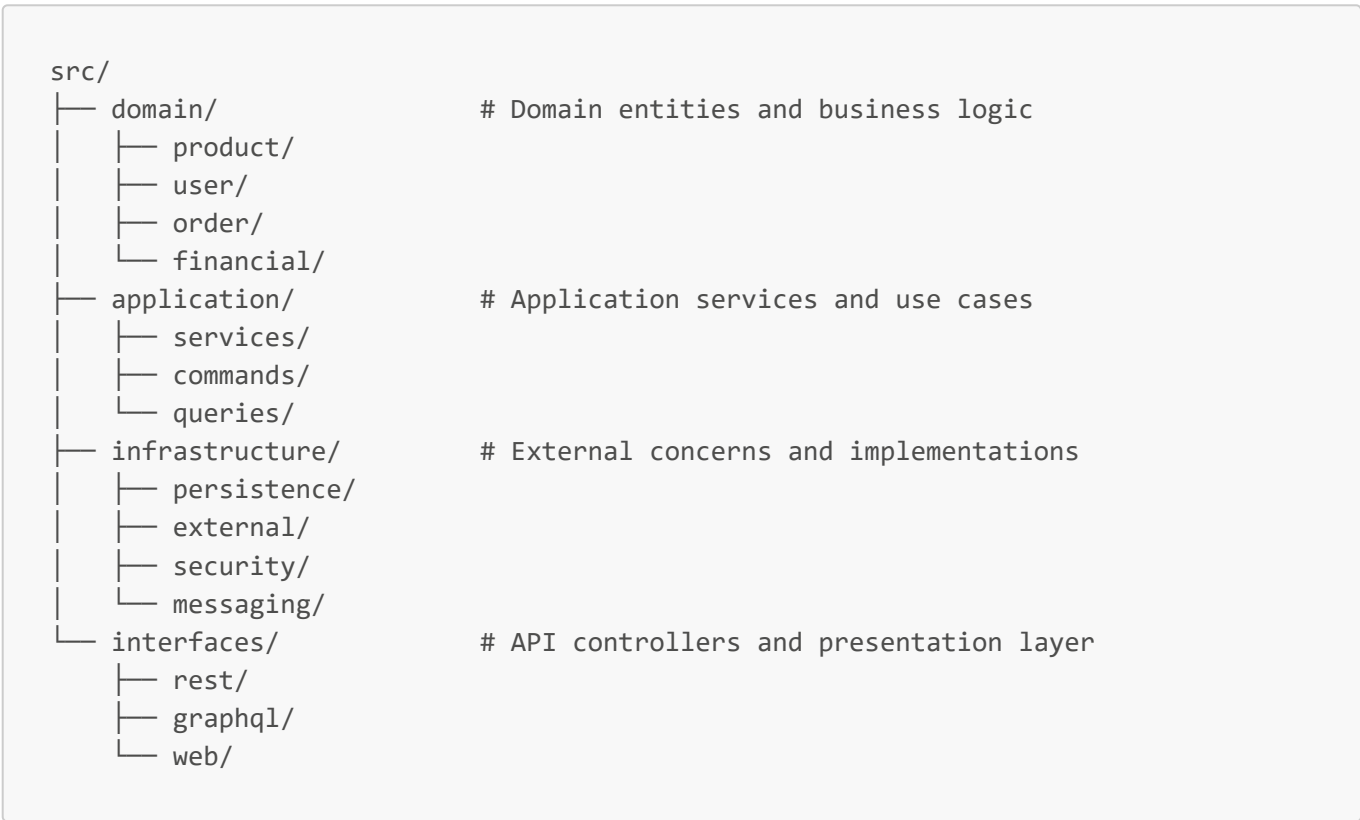
Infrastructure Components

- **Container Orchestration:** Kubernetes for scalable deployment
- **Service Mesh:** Istio for service-to-service communication
- **Database:** PostgreSQL with read replicas for performance
- **Cache:** Redis for session management and caching
- **Message Queue:** RabbitMQ for asynchronous processing
- **Storage:** Object storage for product images and documents

4. Development View

The development view outlines the code organization, build processes, and development workflows.

Project Structure



Development Workflow

1. **Feature Development:** Feature branches with pull request reviews
2. **Testing:** Unit tests, integration tests, and end-to-end tests
3. **CI/CD:** Automated build, test, and deployment pipelines
4. **Code Quality:** Static analysis, code coverage, and security scanning

Technology Stack

Frontend

- **Framework:** React.js with TypeScript
- **State Management:** Redux Toolkit

- **UI Library:** Material-UI or Ant Design
- **Build Tool:** Vite or Webpack

Backend

- **Runtime:** Node.js with TypeScript
- **Framework:** Express.js or NestJS
- **Database:** PostgreSQL with TypeORM
- **Cache:** Redis
- **Message Queue:** RabbitMQ

DevOps

- **Containerization:** Docker
 - **Orchestration:** Kubernetes
 - **CI/CD:** GitHub Actions or GitLab CI
 - **Monitoring:** Prometheus + Grafana
 - **Logging:** ELK Stack (Elasticsearch, Logstash, Kibana)
-

Design Decisions

1. Domain-Driven Design (DDD)

Decision: Adopt DDD principles for complex business logic **Rationale:**

- Clear separation of business concerns
- Improved maintainability and testability
- Better alignment with business requirements
- Facilitates team collaboration

Implementation:

- Bounded contexts for each domain (Product, User, Order, Financial)
- Rich domain models with business logic encapsulation
- Domain services for complex business operations

2. Clean Architecture

Decision: Implement Clean Architecture for dependency management **Rationale:**

- Independence of frameworks and external concerns
- Testability and maintainability
- Clear dependency flow from outer layers to inner layers
- Technology agnostic core business logic

Implementation:

- Domain layer (innermost) with no external dependencies
- Application layer with use cases and application services
- Infrastructure layer for external concerns

- Interface layer for user interaction

3. Microservices Architecture

Decision: Start with modular monolith, evolve to microservices **Rationale:**

- Reduced initial complexity and operational overhead
- Easier development and debugging
- Gradual evolution based on business needs
- Team autonomy and technology diversity

Implementation:

- Clear service boundaries aligned with domain contexts
- API Gateway for external communication
- Service mesh for internal communication
- Shared database initially, evolve to database per service

4. Event-Driven Architecture

Decision: Implement event-driven patterns for loose coupling **Rationale:**

- Decoupled service communication
- Improved scalability and resilience
- Better handling of asynchronous operations
- Audit trail and business process tracking

Implementation:

- Domain events for business state changes
- Event sourcing for critical business processes
- Message queues for reliable event delivery
- Event store for audit and replay capabilities

Quality Attributes

Performance

- **Response Time:** < 200ms for API calls, < 2s for page loads
- **Throughput:** Support 1000+ concurrent users
- **Scalability:** Horizontal scaling with load balancing

Reliability

- **Availability:** 99.9% uptime
- **Fault Tolerance:** Graceful degradation and circuit breakers
- **Data Consistency:** ACID properties for critical operations

Security

- **Authentication:** Multi-factor authentication
- **Authorization:** Role-based access control (RBAC)
- **Data Protection:** Encryption at rest and in transit
- **Compliance:** GDPR and PCI DSS compliance

Maintainability

- **Code Quality:** High test coverage (>80%)
 - **Documentation:** Comprehensive API and code documentation
 - **Monitoring:** Real-time observability and alerting
-

Risk Assessment

High Risk

1. **Data Security:** Implement comprehensive security measures
2. **Scalability:** Design for horizontal scaling from the start
3. **Integration Complexity:** Plan for third-party service integration

Medium Risk

1. **Performance:** Implement caching and optimization strategies
2. **Data Migration:** Plan for database schema evolution
3. **Team Skills:** Provide training on new technologies

Low Risk

1. **Technology Selection:** Use proven, well-documented technologies
 2. **Compliance:** Follow industry standards and best practices
-

Evolution Strategy

Phase 1: Foundation (Months 1-3)

- Core domain models and basic CRUD operations
- User authentication and authorization
- Basic product catalog and shopping cart

Phase 2: Enhancement (Months 4-6)

- Order processing and payment integration
- Advanced search and filtering
- Basic reporting and analytics

Phase 3: Scale (Months 7-12)

- Microservices migration
- Advanced features (recommendations, personalization)
- Performance optimization and monitoring

Phase 4: Innovation (Months 13+)

- AI/ML integration
- Mobile applications
- Advanced analytics and business intelligence

This architecture overview serves as the foundation for all development decisions and should be reviewed and updated as the system evolves.